

Avoiding Unnecessary Caching with History-Based Preemptive Bypassing

Arthur M. Krause
Institute of Informatics
UFRGS

Porto Alegre, Brazil
arthur.krause@inf.ufrgs.br

Paulo C. Santos
Institute of Informatics
UFRGS

Porto Alegre, Brazil
pcssjunior@inf.ufrgs.br

Philippe O. A. Navaux
Institute of Informatics
UFRGS

Porto Alegre, Brazil
navaux@inf.ufrgs.br

Abstract—Cache memories can account for more than half of the area and energy consumption on modern processors, which will only increase with the current trend of bigger on-die memories. Although these components are very effective when the access pattern is cache-friendly, cache memories incur extra and unnecessary latencies when they cannot serve the data, which adds to significant energy wastes when data that is never reused is placed on them. This work introduces HBPB, a mechanism that detects whether a memory access is cache-friendly or not, allowing the bypass of the cache for accesses that are not known to be cache-friendly. Our approach allows the processor to quickly detect when caching accesses is inadequate, improving overall access latency and reducing energy waste and cache pollution. The presented solution achieves reductions of up to 28.6% in energy consumption and 19.5% in latency for SPEC applications, and further improvements in power and performance across various workloads.

Index Terms—Cache Memory, Energy Efficiency, Cache Bypassing

I. INTRODUCTION

As processors keep getting faster, cache memories play a vital role in reducing average memory access latencies, becoming even more critical with each new generation of processors. The current trend in processor design is to increase the number of cores and the size of the caches. However, by increasing the number of processing cores, the competition for space and bandwidth in shared cache memories also increases, which impacts the performance and efficiency of these memories due to over-utilization, cache pollution, and trashing. With bigger caches, energy consumption and access latencies also tend to increase. Thus, the need for more intelligent management of these resources is of great and growing importance.

Not every memory access can be helped by caching. In fact, for many accesses, caching the data provokes only extra latency and energy consumption while also replacing other potentially useful data, which will then have to be fetched again from a slower level of the memory hierarchy. This results in once more increased latency and energy consumption indirectly. In such a way, when cache-unfriendly memory access are performed and passed through the cache hierarchy,

not only they suffer from increased latency and unnecessary energy waste, they also pollute the caches, hampering their proper operation for cache-friendly accesses [1].

Current processors have limited implementations of bypassing instructions, which are not well documented for *load* operations and can have unpredictable behavior that depends on specifics of the implementation [2]. Also, instructions of this kind cannot be used with fine granularity and require the programmer to employ them properly. Compilers do not automatically employ these instructions since they would have to know at compile time that an instruction always accesses non-temporal data. Lately, progress has been made with cache replacement policies that can tolerate streaming access patterns and mitigate the cache pollution generated by them, which are effectively implemented in current processors available in the market [3]. However, such methods still perform caching of cache-unfriendly data, not avoiding the massive energy waste that they represent. For current processors, the only way to effectively bypass the cache is through non-temporal instructions or by defining a whole region of memory as non-cacheable with the granularity of a page.

Many researchers have addressed this problem through varied approaches [4]–[9]. However, none of the existing works present a proposition to bypass both the memory requests as well as the data fills, on all levels of the cache hierarchy, while allowing spatial locality and short-term temporal reuse to happen still. Bypassing only for the LLC can be insufficient as unnecessary fills to the higher-level caches can cause evictions that provoke victim fills or write-backs to the lower levels of the cache. In this context, we propose a History-Based Preemptive Bypassing (HBPB) mechanism, with the goal of bypassing the cache memory for every access that is not known to be cache-friendly and learning which accesses could benefit from caching.

This paper is organized as follows: Section II presents the motivation for bypassing the caches. In Section III, the state-of-the-art in cache bypassing is briefly described. Section IV presents HBPB, our proposed implementation of cache bypassing in hardware. In Section V, we show that HBPB improves performance and energy efficiency across different hardware configurations. Section VI brings a conclusion to this work.

This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001. By CNPq/MCTI/FNDCT - Universal 18/2021 under grants 406182/2021-3, and by Petrobras 2020/00182-5.

II. CACHE BYPASSING MOTIVATION

Cache memories are a powerful mean to mitigate the memory-wall problem [10], highlighted by the performance gap between logic and main memory technologies. Usually organized in a multi-level hierarchy, with smaller and faster levels closer to the processing cores, they reduce the average memory access latency by storing data that is likely to be requested by the processor in the near future. The core functionality of the cache is exploring the spatial and temporal locality of memory accesses. The use of cache lines covers spatial locality: every time an access is made, 64 bytes of data - a cache line - are fetched. This way, if the next accesses are for neighboring data, they have already been fetched, and the latency is reduced. Temporal locality means that data that has been recently requested tends to be requested again in the near future. The caches leverage the temporal locality of memory accesses by storing data that has been requested in the near past, evicting data that has not been used in a long time when new lines need to be stored.

However, caches are not effective when the access pattern has no temporal locality in regards to cache lines. For example, when the application searches through a linked list, and nodes are not repeated, no data can be serviced by the cache. In this case, regular cache management dictates that the processor must first check the caches for the presence of the needed data and then store the data coming from main memory in the caches, in case it is needed in the future, which never happens. The actions of waiting for the cache answer before requesting the data to main memory and filling this data in the cache represent both an unnecessary overhead. If the request could bypass the cache, many cycles of waiting would be avoided, and if the data could bypass the cache, a significant amount of energy would be saved. Also, by not storing unnecessary data into the cache, cache pollution - replacing lines with reuse potential with lines with no reuse potential - could be mitigated, which in turn would reduce cache misses, saving on latency and energy again.

Current processors offer the functionality of *load*, *store* and *prefetch* instructions with a *non-temporal hint*. *Non-temporal stores* are well established due to the need for offloading data to other devices with short latencies and are used mainly on drivers and kernel code. Also, the possibility of using write combining - buffering many stores to neighboring locations before issuing a single write operation to memory - makes non-temporal stores even more useful. However, for *load instructions*, functionality is much more restricted. On ARM, the LDNP instruction performs a load of a pair of values into two registers with a non-temporal hint. What the processor does with this hint is not well specified. On x86, non-temporal load instructions have been introduced with SSE 4.1. When these instructions are used interchangeably with regular load instructions, caching is usually done anyway, rendering their employment almost useless [2]. For proper bypassing of the caches to occur, the memory region that is being accessed must be configured as *write combining*, reducing the freedom

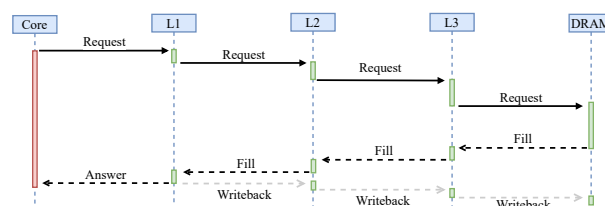


Fig. 1: Sequence diagram of a cacheable memory access through a traditional memory hierarchy.

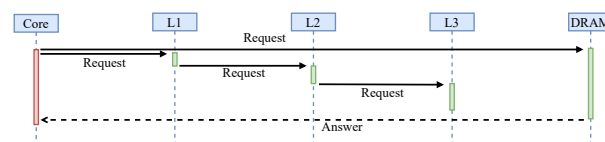


Fig. 2: Sequence diagram of a non-temporal memory access through a traditional memory hierarchy.

for the programmer or compiler to employ them.

In addition to not having a transparent and dependable implementation, these instructions only exist for Single Instruction Multiple Data (SIMD) operations and have to be used explicitly. For example, while a regular arithmetic instruction can perform a memory access, for this access to be made non-temporal, a non-temporal SIMD load has to be performed for the desired data, the value has to be copied from the SIMD registers to regular registers, and then the arithmetic operation can be done. Also, as the memory region has to be defined as *uncacheable* at the allocation, this limits the flexibility to decide between bypassing the cache or not depending on the state of the system or bypassing only some parts of the data on a data structure. Consequently, detecting which instructions could perform non-temporal accesses at run-time is of great value.

Figure 1 shows the sequence diagram of a memory access that misses in all levels of cache and is serviced by the main memory, considering all caches are inclusive. The request crosses all cache levels and prompts fills and evictions on every level. If the evicted line has been modified, a Write Back (WB) to the next level of the memory hierarchy must be done. The data is requested to the main memory only after it is confirmed that it is not present in the caches. Figure 2 shows how a non-temporal memory access is performed. The request goes directly to the main memory, and the caches are checked in parallel. Since the request has arrived in the main memory much earlier, the answer arrives earlier. If the data happened to be in the caches, the data from the main memory is discarded. No fills are done, so no evictions or Write Backs are necessary.

Figure 3 shows the different latencies involved in a main memory access both in a regular access and in a bypassed access. Since the physical address generation is usually done in parallel with the L1 cache check, it can not be avoided by the non-temporal access. The bypassed access takes 45 fewer

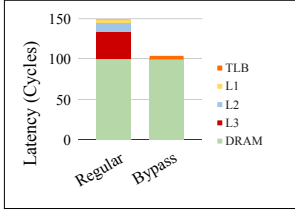


Fig. 3: Latencies in an access.

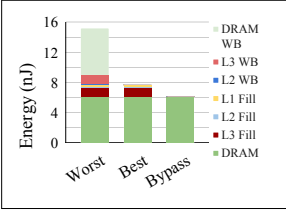


Fig. 4: Energy costs of an access.

cycles than usual, being 30% faster, assuming the DRAM latency for a row buffer hit.

Figure 4 shows the energy costs of an access to main memory in the worst case, in which all cache evictions are from dirty lines and Write Backs are prompted on all levels. In the best case, no Write Backs are done, and so the only energy costs are from the DRAM read and cache fills. When the access is non-temporal, the only energy cost is from reading the DRAM. With that in mind, a non-temporal access can save up to 60% of the dynamic energy in comparison with a regular access. Also, considering the latency is reduced, it further saves on static energy. It is crucial as well to note that saving on power inside the processor package is more beneficial due to thermal dissipation factors.

III. RELATED WORK

Improving the memory system efficiency is of major importance and is the goal of a growing proportion of the research in computer architecture. From complete overhauls of their organization to minor policy tweaks, the caches are the usual target for enhancements in the memory hierarchy. A usual approach is to improve the cache replacement policy, which means changing the way that a cache block is selected for eviction when a new block must be stored [11]–[16].

Another approach is to bypass the caches when serving data to the processor. Many works suggest bypassing only the LLC, with varied methods [5], [6], [9], [17]–[21].

Other authors propose bypassing cache levels where there is no perceived benefit for a given data set [7], [22], [23], while Egawa et. al go further and suggest completely power gating a cache level if it is not efficient [8].

Recently, proposals that aim to predict in which level of the memory hierarchy a load will hit in order to anticipate the requests to the appropriate level have gained great notoriety [24], [25].

Most of the previous work is, to some degree, very conservative with bypassing. Most of the time, they suggest bypassing only the LLC or being exceedingly selective when deciding to bypass or not. In this context, our work aims to complement the state-of-the-art with a proposal of a mechanism that bypasses all the caches preemptively in both ways, requests and data, and show that it can be done in hardware at runtime, using a straightforward heuristic.

The main focus of this work is to analyze the impact of bypassing all the cache levels, preemptively, on both ways

while offering a small buffer for spatial reuse to still occur. Employing a separate prefetcher on this buffer to target the non-temporal accesses is also an important contribution of our work. We also propose a heuristic to select the lines to be bypassed.

IV. HISTORY-BASED PREEMPTIVE BYPASSING

In order to achieve an online conversion of regular memory accesses into non-temporal accesses, we propose the History-Based Preemptive Bypassing (HBPB) mechanism. The mechanism has two main goals:

- Increase performance by avoiding cache latencies for cache-unfriendly access;
- Reduce cache pollution and increase energy efficiency by not filling data from cache-unfriendly accesses into the caches.

The mechanism works by storing the history of recent memory accesses and classifying instructions into cache-friendly or not. HBPB assumes every instruction is not cache-friendly unless it has been detected as such. Thus, it aggressively tunes accesses from instructions not known to be cache-friendly into non-temporal (bypassed) memory accesses, preemptively bypassing all the cache levels.

A. HBPB Architecture

The mechanism requires three major additions to the architecture, as displayed in Figure 5. First, a table that stores identifiers of instructions is required. Each entry has the Program Counter (PC) as a key and a saturating counter as a value. This table is called the Classified Instructions Table (CIT). In order to reduce the storage needed, every PC can be hashed, and the mechanism then only sees this hash.

The mechanism also needs an Access History Table (AHT), which stores memory addresses and the identifier of the instruction that generated the last memory accesses to this address. This table needs to be as big as the number of lines in the caches and keep their ordering. It can be a replica of the LLC for better efficiency, storing the PC hash of the instruction that generated the access instead of the cache block. This is analogous to an L3 cache simulator inside the processor.

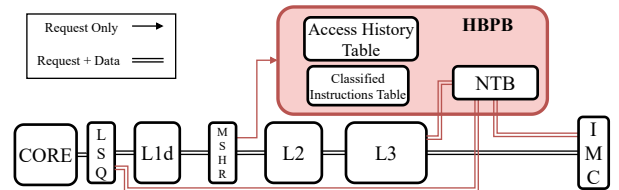


Fig. 5: HBPB architectural additions.

We also need a Non-Temporal Buffer (NTB) structure to hold the data requested by bypassing instructions. This buffer allows for data that have reuse only in a short distance to be serviced by this buffer, avoiding polluting the caches. The NTB allows the mechanism to sample only the L1d new misses, drastically reducing its overhead. The NTB acts as

a parallel L1d, avoiding filling the L1 with non-temporal data and trickling unnecessary Write Backs through the cache hierarchy.

HBPB receives requests from the L1 and Last Level Cache, and serves data from the NTB to the LLC and directly to the core Load-Store Queue (LSQ). It requests, receives, and sends data to the DRAM.

B. HBPB Operation

The workings of the mechanism are depicted as a flowchart in Figure 6. When a memory accessing instruction is executed,

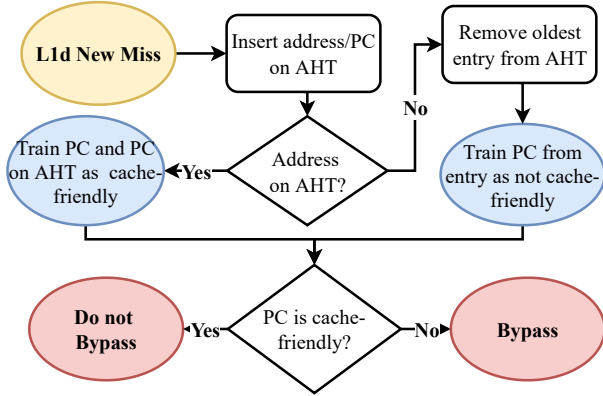


Fig. 6: HBPB decision flowchart.

and the access provokes a new miss on the L1 data cache, the processor sends the access information to HBPB. The mechanism will then decide if this memory access should be bypassed, and use the data from the access to train its future decisions.

1) *Decision*: When HBPB is requested to decide if an access should be bypassed or not, it checks for the presence of the instruction identifier on the CIT and the value of its counter. If the counter is above the threshold, the mechanism signals to the L1d that the access should not be bypassed. If the instruction is not present on the table or its counter is below the threshold, the L1d is informed that this access should be bypassed.

2) *Training*: The mechanism checks for the presence of the memory address on the AHT. If the address is on the table, it means the data has been reused, and the instructions that requested it are cache-friendly. Thus, the mechanism increments the counters for both the current instruction and the instruction that initially brought the data into the AHT on the CIT. If there is no entry for the instructions on the CIT, they are added with the counter set above the threshold.

If the address is not present in the AHT, it is inserted, and one element of the table is evicted, according to the AHT replacement policy, which is the same as the LLC. The instruction that brought the evicted element has accessed a dead line, and so its counter is decremented on the CIT if present. The value decremented from the counter must be smaller than the value added on hits so that instructions are

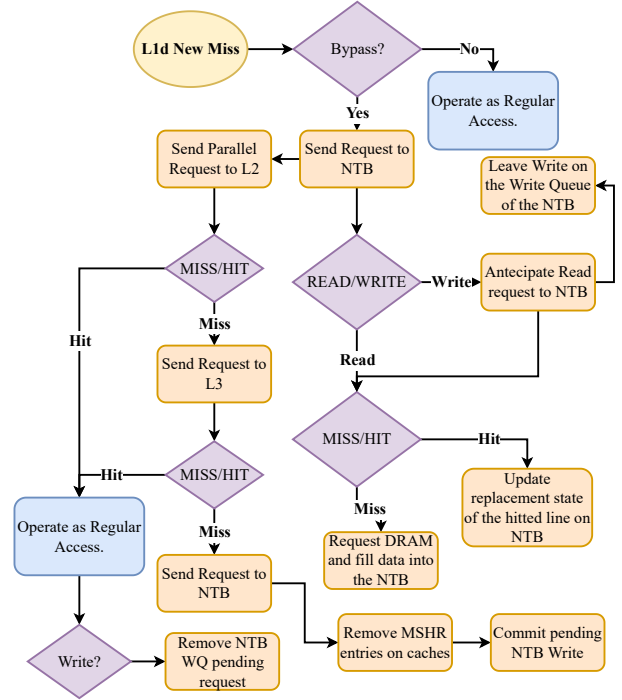


Fig. 7: The path of the request through the memory subsystem.

not excessively penalized for a short burst of evictions of its lines from the AHT.

If the address misses in the AHT and the instruction is present on the CIT with its counter saturated negatively, HBPB understands that this instruction is strongly cache-unfriendly and does not add the address to the AHT. This reduces the number of writes to the structure and makes it a better representation of the cache without the data that is very likely to be bypassed. In this situation, the mechanism still adds the address to the table with a 1/32 chance to allow instructions that eventually change behavior to be reclassified.

C. Data and Request Paths With HBPB

The paths of the memory requests through the system are depicted in Figure 7, and Figure 8 illustrates the paths of the data. When a miss occurs in the L1 data cache for an address that is not already present on the Miss Status Holding Registers (MSHR), it means that a new request needs to be propagated through the memory hierarchy, usually in the form of an L2 request. With the usual operating scheme of the caches, the requests are propagated in only one path: New L1 misses are requested to the L2; New L2 misses are propagated to the L3, for which new misses generate a new DRAM request. The memory that has the data then sends the data back to the memory that asked for it, trickling the data back to the processor core.

We propose an alternative way for requests to arrive in the DRAM. When the HBPB orders an L1 miss to be bypassed,

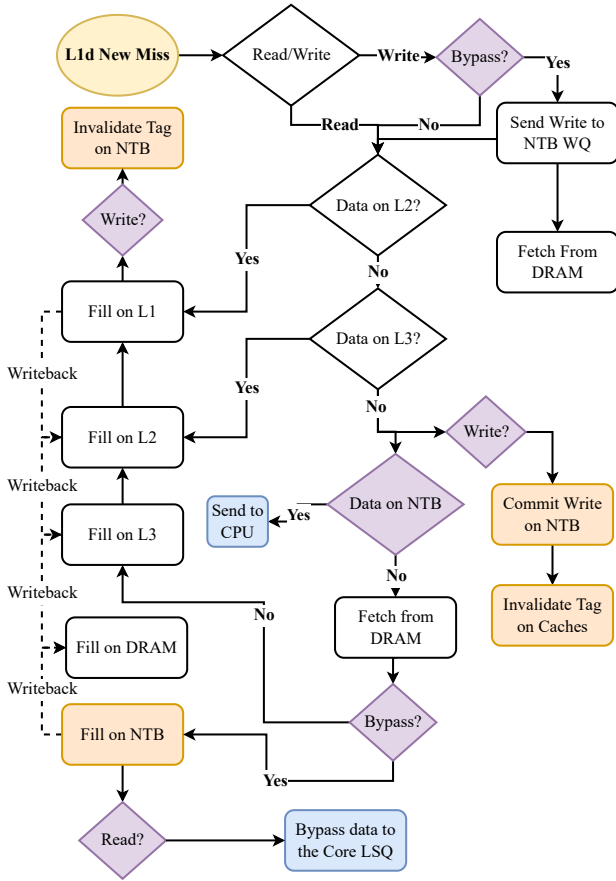


Fig. 8: The path of the data through the memory subsystem.

a new request is created and forwarded to the NTB, which will then propagate the request to the DRAM if it results in a new miss. The lower level caches still have to be checked in parallel because it is possible that the access is a cache hit. When the parallel cache query is confirmed as a miss, the NTB serves the data to the processor, and the pending misses in the cache MSHRs are deleted. If the NTB has a copy of the data, it can serve it to the processor immediately. Having a write-invalidate coherence protocol guarantees that the NTB has the most up-to-date version of the data.

Write operations that miss on the L1 cache generate a Read For Ownership (RFO) request that propagates to the other caches. Bypassed RFOs also propagate the write operation to the NTB, but they can be committed only when the miss on all the caches is confirmed. If the parallel request to the caches ends up resulting in a hit, this pending write is canceled, and the data read from memory is invalidated. When the write is committed in the NTB, all lines storing this address in the caches must be invalidated, following the write-invalidate protocol. To maintain data consistency, the LLC has to confirm that an address is not present in the NTB before accepting a new line from the DRAM, in case it has been written by a

bypassed write.

Non-Temporal accesses are deployed to both the caches and the main memory. If the mechanism has mistakenly classified the instruction as not cache-friendly and the request hits in the caches, the access operates regularly, the PC is trained as cache-friendly on the CIT, and the data from the NTB is not read by the access.

The most significant benefit of inserting only data that is likely to be cache-friendly in the caches is that it does not require extensive training before deciding to bypass a cache line. On existing mechanisms, the decision to bypass data from an instruction is only made after said instruction has already flushed the entire cache. By doing this more aggressive bypassing, we can avoid pollution and energy waste from shorter bursts of accesses, leaving the cache available for data that is proven to be reusable.

D. HBPB Hardware Overhead

The most significant hardware overhead incurred by HBPB is the AHT. This structure must store one entry for every cache line available for the processor. Each entry is comprised of an instruction identifier and a memory address. We do not need to store the whole 64 bits of the Program Counter to identify each instruction inside the mechanism. For better area efficiency, we identify each instruction as a 16-bit hash of its PC. Since the AHT is a clone of the Last Level Cache, the memory address is stored as a combination of index and tag, as in the caches. On a configuration with a 1 MB, 16-way set associative Last Level Cache with 64-Byte cache lines, the tag field of a cache block is 48 bits long.

This means the AHT would need to store 64 bits for every cache line available on the processor LLC, while the LLC needs 560 bits for the tag and data fields. The AHT then needs an area equal to 11.4% of the LLC. Such an implementation implicates that the AHT would require 131 KB of extra storage. Larger cache sizes would need an AHT proportionally smaller because there are fewer bits required for the tag, given that the associativity remains the same.

The other major overhead from HBPB is the NTB, which has the same size as the L1 data cache. In this case, 32 KB for the data array and 3264 Bytes for the tag array.

The CIT must store the instruction identifier with its saturated counter. The mechanism requires this structure to have low read latency since the decision to issue a regular or non-temporal access has the window of time between the instruction decode and its placement on the LSQ. Thus, a hash table of counters indexed by the hash of the Program Counter is an appropriate solution for it. We observed that storing the state of as few as 128 instructions per core yields indistinguishable results compared to an infinite table that stores all instructions.

V. EVALUATION

We implemented our mechanism in ChampSim [26], a cycle-accurate, trace-driven x86 simulator. Full details of the modeled system are present in Table I.

We tested three different versions of HBPB to better understand the practical effects of each of the proposed modifications:

- **HBPB-R**: Bypasses only the requests. Cache fills are done normally;
- **HBPB-F**: Bypasses only the fills. No parallel requests are issued;
- **HBPB-RF**: Bypasses both requests and fills.

For fairness in the results, we compared the HBPB configurations with the LLC reduced, to offset the extra overheads from HBPB with a significant margin. The LLC was reduced from 16-way to 13-way, while keeping the same number of sets. This reduces the cache capacity by 18.75%. The latency and energy consumption of the smaller cache was kept the same in order to keep the number of changes at a minimum and avoid giving any unfair advantages to the HBPB configurations. We compare the effects of the mechanism with a baseline configuration without HBPB and a bigger LLC.

TABLE I: Modeled system.

Component	Configuration
Core	3.4 GHz, 64/36-entry Load/Store Queue
L1 Instruction Cache	32KB, 8-way, 4 cycles latency
L1 Data Cache	32KB, 8-way, 4 cycles latency
L2 Cache	256KB, 8-way, 11 cycles latency
Baseline L3 Cache	1MB, 16-way, 34 cycles latency
Non-Temporal Buffer	32KB, 8-way, 4 cycles latency
HBPB L3 Cache	832KB, 13-way, 34 cycles latency
DRAM	DDR3, 1600 MT/s, tCAS = 10 Dual Channel/Ranks, 8 Banks

In order to verify that the benefits from HBPB translate to real applications, we tested the mechanism with some applications from the memory intensive benchmarks from SPEC CPU 2017 [27], using the traces generated for the ML-Based Data Prefetching Competition [28], with the biggest workload sizes for each application. In this section, only the effects of HBPB on the ED²P of the SPEC applications are presented, due to space constraints.

We use Cacti [29] to estimate the energy consumption of the caches and, to the best the tool can provide, the energy overhead for the HBPB structures. The values used in the evaluation are shown in Table II. Energy consumption values are a product of the number of reads and writes to the memories and their respective energy costs. Each access to HBPB incurs in a read and a write to the AHT, except when the information is not filled. In this case, only a read is accounted for.

TABLE II: Energy values used on the evaluation.

Cache	Read Energy (pJ)	Write Energy (pJ)
L1	199	202
L2	169	175
L3	925	936
NTB	199	202
AHT	50	87

We present the effect of the mechanism on the cycles, cache dynamic energy, and Energy-Delay-Squared Product (ED²P) of the applications. The ED²P is calculated as a product of the square of the number of cycles and the dynamic energy of the caches, including the HBPB structures. The ED²P allows the balance between performance and power to be synthesized in a single value. We chose to have the cycles squared to give more weight to the performance part of EDP, which is the usual approach when employing this metric. As in the cycles and energy, the smaller the ED²P, the better.

A. Experimental Results

The resulting effects of HBPB on the ED²P of the SPEC 2017 benchmarks were positive except for two occasions, as shown in Figure 9. On *gcc*, since HBPB-F decreased the performance and did not improve the energy consumption significantly, the ED²P was increased by 4.1%. On *mcf*, since HBPB-R increased the energy consumption and decreased the total cycles by a smaller margin, the ED²P ended up being raised by 5.3%. HBPB-RF never deteriorated the ED²P. Apart from *cactuBSSN*, where the mechanism had nearly no effect, and *fotonik3d*, *wrf* and *omnetpp*, where the decreases in ED²P were of 2.9%, 2.3%, and 2.4%, respectively, HBPB-RF yielded sizeable benefits when considering the ED²P as the desired metric.

On *gcc*, even if not as great as the reduction by HBPB-R, of 24%, HBPB-RF reduced the ED²P by nearly 7%, simply due to the speed-up achieved by bypassing the requests. On *roms*, the ED²P reduction yielded by HBPB-R was also greater than the one by HBPB-RF, since the speed-up with the first was bigger. HBPB-R reduced ED²P by 12.6%, while HBPB-RF reduced it by 5.8%. On *lbm*, *xalancbmk* and *mcf*, since HBPB-RF improved both performance and energy efficiency, the reductions in ED²P are much more substantial, being 15.8%, 38.4% and 53.7% respectively. On average, HBPB-RF reduced ED²P by 16.6%.

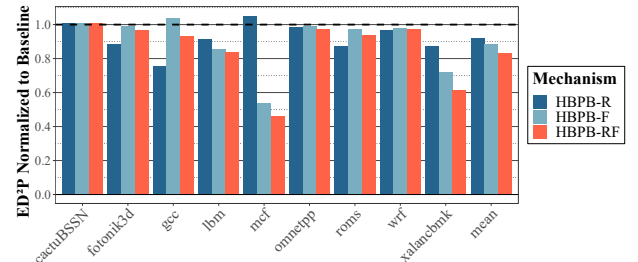


Fig. 9: Effect of HBPB on the ED²P for the SPEC CPU 2017 benchmarks.

Table III summarizes the key impacts of HBPB on the SPEC applications' performance and energy efficiency. For some of the applications, HBPB-RF massively improved energy efficiency and performance while never affecting any application negatively by more than 1%.

TABLE III: Summary of the percentual reductions in Cycles and Cache Energy obtained with HBPB on the SPEC CPU 2017 applications.

	<i>cactuBSSN</i>	<i>fotonik3d</i>	<i>gcc</i>	<i>lbm</i>	<i>mcf</i>	<i>omnetpp</i>	<i>roms</i>	<i>wrf</i>	<i>xalancbmk</i>	<i>mean</i>
Cycles (R)	0.0	7.0	18.1	5.4	0.4	1.3	8.4	2.4	9.1	5.9
Cycles (F)	0.0	0.5	-2.1	0.7	12.3	0.0	0.1	0.1	6.3	2.1
Energy (F)	-0.9	0.0	0.3	12.9	29.1	1.3	2.6	1.6	17.6	7.7
Cycles (RF)	0.0	1.6	3.4	2.4	19.5	0.6	2.0	0.5	14.1	5.1
Energy (RF)	-0.9	-0.2	0.3	11.7	28.6	1.1	2.0	1.3	16.6	7.3

B. Proportion of bypasses

The plot in Figure 10 shows the decisions made by HBPB-RF on incoming requests arrived after an L1 miss. It helps in understanding the effects of the mechanism on the applications. The accesses are presented as a percentage of the total program instructions.

On *lbm*, *xalancbmk*, *gcc*, and *mcf*, applications that were sped-up by the mechanism, a sizeable portion of the Load instructions were bypassed, reducing their latency and improving the performance. On *lbm*, the improvements in energy efficiency were much greater than those in performance, which is explained by most of the accesses from this application that suffered interference from HBPB being Stores. Since Stores are not as relevant for program performance, the effect of bypassing them is more perceivable on the cache energy consumption.

Some applications were not affected by the mechanism for different reasons. On *cactuBSSN* for example, the HBPB decided to make regular accesses for practically all the instructions. On *wrf*, *omnetpp* and *fotonik3d* there were simply not enough L1 misses for HBPB to act on. Meanwhile, *mcf* has more than 16 L1 misses by 100 instructions, and HBPB-RF can leverage this behavior to bypass many accesses and improve efficiency.

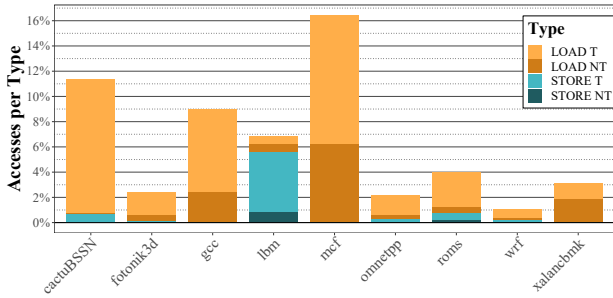


Fig. 10: Frequency of each type of access and bypass decision for the SPEC 2017 benchmarks with HBPB-RF.

C. Static Reuse Distance Analysis

We performed an analysis of the memory access traces of the applications to better understand why HBPB was able to improve performance in some programs but not in others. ChampSim was modified to provide an output of all the

memory accesses, with their instruction PC and the memory address of the data.

With this data, we calculated the reuse distance - the amount of memory accesses to unique addresses between two references to the same address - for each access and grouped them by instruction. We group the reuse distances into five buckets:

- **MSHR**: When the reuse distance is between 0 and 2. We assume that requests to the same address that are close to each other are merged in the buffers;
- **L1**: When the reuse distance is greater than the MSHR but smaller than the size of the L2 cache;
- **L2**: When the reuse distance is greater than the L1 cache but smaller than the size of the L3 cache;
- **L3**: When the reuse distance is smaller than the capacity of the caches but greater than the capacity of the L2 cache;
- **Miss**: When the reuse distance is greater than the capacity of the caches.

We calculated both the forward distance - the number of unique addresses referenced between a given reference and the next reference to the same address - and the backward distance - the number of unique addresses accessed between a given reference and the last reference to this address. For example, if the backward distance for an access is inside the L3 bucket, we expect this reference to result in an L3 hit. If the forward distance for this access is into the Miss bucket, we expect that the next reference to this address will be a cache miss or that it is never going to be accessed again.

From the line in the middle, we plot the reuse distances for each of the 50 most frequently occurring instructions of each application, spread across the x-axis ordered by frequency. The bars are colored regarding the frequency of each reuse distance bucket. The forward reuse distances are stacked above the line, while the backward distances are stacked below the line.

We combine this data with the simulation results obtained when running the applications with HBPB-RF. The instructions that have been bypassed are annotated with a color code representing the ratio of bypasses performed for the instruction. Green lines indicate that the instruction was bypassed more than 90% of the time, yellow means the instruction was bypassed between 50% and 90% of the time, while red lines indicate that the memory accesses from the instruction were bypassed between 10% and 50% of the time. Store instructions the dot above the bars, while loads are annotated at the bottom.

The ideal application for HBPB has many misses and instructions that are clearly offending in a manner that they are responsible for bringing dead data into the cache or always causing misses when accessing the memory.

Due to space constraints, we show the plots for only three of the applications that were used. Two that were vastly improved by the mechanism: *mcfl* and *xalancbmk*; and *omnetpp*, an application that was not deeply affected by HBPB.

1) *mcfl*: The distribution of the reuse distances of *mcfl*, as shown in Figure 11 highlights the potential of this application to benefit from reduced cache pollution. The three most frequent instructions from *mcfl* are responsible for the majority of the memory accesses, with the fourth and fifth most frequent instructions also heavily accessing the memory. The most frequent instruction is heavily cache-unfriendly, and HBPB correctly bypasses nearly all the accesses from this instruction. The second and third instructions are deeply cache-friendly, and HBPB does not bypass their accesses, while the fourth and fifth most frequent instructions are also almost always bypassed. All of them are loads, which makes their bypassing more contributing to the final performance.

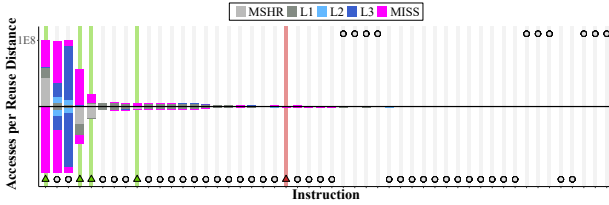


Fig. 11: Frequency of reuse distances for the 50 most frequently occurring instructions for *mcfl*.

The bypassed instructions that would otherwise pollute the cache with their data no longer do it. This leaves more space in the caches for the also very frequent cache-friendly instructions. That is the reason for the performance being improved mainly by HBPB-F on this application. HBPB-F can display all its potential in *mcfl*, where it speeds up the application by avoiding cache pollution, saves energy by not filling unnecessary data into the cache, and avoids fills by not evicting data that has reuse for data that does not.

2) *xalancbmk*: On *xalancbmk*, the majority of the accesses is distributed among few instructions, of which a significant portion exhibits long reuse distances, as shown in Figure 12. These instructions, however, also show some sort of temporal reuse inside the L2 and L3 ranges, making caching them sometimes useful. HBPB then bypasses the accesses from these instructions the majority of the time, but not always. If in a specific program phase these instructions show cache-friendliness, they are not bypassed.

3) *omnetpp*: On *omnetpp*, for which the reused distances are shown in Figure 13, the most frequently occurring memory accessing instructions display short reuse distances that frequently fall into the L1, L2 and L3 range. There are no clearly non-temporal instructions. Only a very small amount of the least frequent instructions are bypassed.

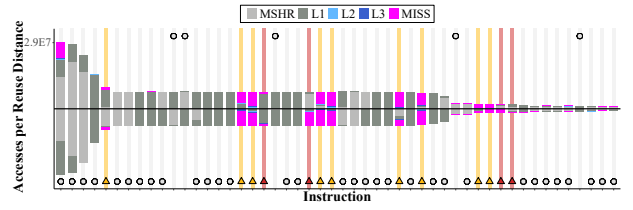


Fig. 12: Frequency of reuse distances for the 50 most frequently occurring instructions for *xalancbmk*.

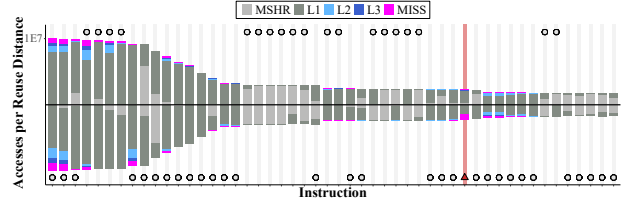


Fig. 13: Frequency of reuse distances for the 50 most frequently occurring instructions for *omnetpp*.

This analysis allows us to understand more deeply how HBPB works, and proves how the instruction reuse pattern is consistent for each instruction. Specially on *mcfl*, it is clear how having predominantly offending instructions coexisting with predominantly cache-friendly instructions allows HBPB-RF to show its strength at its fullest.

D. Sensibility to Memory Configuration

Different hardware configurations change the cache behavior and can impact the effects of HBPB. We tested HBPB with a different cache size and replacement policies and introduced hardware data prefetchers to study their impact on the mechanism.

1) *Bigger Cache Capacity*: With a higher cache capacity, bypassing can become less attractive since the caches have a higher probability of serving the data and being useful. As shown in Figure 14, HBPB is less effective if the L3 capacity is bigger. On *xalancbmk*, the mechanism has no effect with an 8 MB LLC, which is expected since this application's working set fits almost entirely into the L3. Still, HBPB-RF improved ED²P even with bigger LLC capacities.

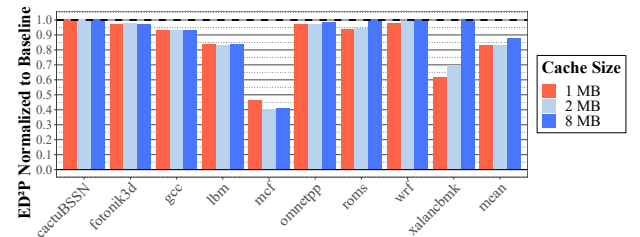


Fig. 14: Effect of HBPB on the cycles for the SPEC CPU 2017 benchmarks with different cache capacities.

2) *Prefetcher*: Data prefetching is a very useful technique employed to reduce the memory access latencies. By anticipating what data the processor may need in the near future, the prefetchers can bring this data closer to the processor, into a higher level cache, for example. If a prefetch is not useful, however, it incurs an unnecessary cache fill, eviction, and potentially the need to re-fetch the data that was evicted, resulting in reduced energy efficiency and performance.

Hardware data prefetchers completely change the pattern of requests between the levels of the memory hierarchy, including the L1 misses, and it is deemed necessary to investigate how prefetching impacts the effects of HBPB. For the cache prefetchers, we utilized a next-line prefetcher for the L1, an IP-Stride prefetcher for the L2, and a next-line prefetcher for the LLC that operates only on regular accesses, not cache checks from bypassed accesses. We also utilized an IP-Stride prefetcher on the NTB that operates only on bypassed accesses.

The effect of the prefetchers on the ED²P strongly varied across the applications, as shown in Figure 15. On *cactuBSSN*, *mcfl*, *omnetpp*, and *xalancbmk*, prefetchers worsened the final ED²P, while on the others they yielded improvements. When compared to the baseline version with no HBPB nor prefetchers, the prefetchers alone reduced ED²P by 31%, and when employed over HBPB-RF, ED²P was reduced by 34%. When comparing both versions with prefetchers, HBPB reduced ED²P by 4.5%.

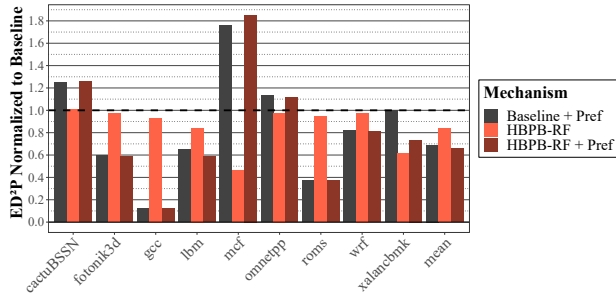


Fig. 15: Effect of HBPB on the ED²P for the SPEC CPU 2017 benchmarks with prefetchers.

Across all the applications and metrics, the best configuration always included HBPB-RF. Table IV shows the best version among the configurations with HBPB-RF, baseline, with and without prefetchers. Versions with results within 0.5% are considered equal. On average, HBPB overperformed the baseline configuration on all metrics, even with hardware prefetchers available. For ED²P, HBPB improved over the baseline configuration in 5 out of 9 applications.

3) *Cache Replacement Policies*: Different cache replacement policies have been developed, aiming to keep data that is more useful in the cache to the detriment of data that is deemed less useful. We test HBPB on three of these policies, employed on the LLC, as well as LRU, which was used previously. The AHT uses the same replacement policy as the LLC.

TABLE IV: Best configuration for each metric for each of the SPEC applications considering prefetchers.

Application	Best Performance	Best Energy	Best ED ² P
<i>cactuBSSN</i>	HBPB/Base + Pref	HBPB/Base	HBPB/Base
<i>fotoni3d</i>	HBPB/Base + Pref	HBPB/Base	HBPB/Base + Pref
<i>gcc</i>	HBPB/Base + Pref	HBPB/Base	HBPB/Base + Pref
<i>lbm</i>	HBPB + Pref	HBPB	HBPB + Pref
<i>mcfl</i>	HBPB	HBPB	HBPB
<i>omnetpp</i>	HBPB/Base + Pref	HBPB	HBPB
<i>roms</i>	HBPB/Base + Pref	HBPB	HBPB/Base + Pref
<i>wrf</i>	HBPB/Base + Pref	HBPB	HBPB + Pref
<i>xalancbmk</i>	HBPB + Pref	HBPB	HBPB
mean	HBPB/Base + Pref	HBPB	HBPB+Pref

- **LRU**: Least Recently Used
- **SHiP**: Signature-based Hit Predictor (PC) [11]
- **SRRIP**: Static Re-Reference Interval Prediction [30]
- **DRRIP**: Dynamic Re-Reference Interval Prediction [30]

Figure 16 shows the impact of HBPB-RF on the ED²P of the applications with the four different replacement policies. Apart from *cactuBSSN*, HBPB-RF always improves ED²P. The performance of HBPB across the different policies did not vary significantly. With DRRIP, SHiP, and to a lesser extent SRRIP, HBPB had a smaller impact on *xalancbmk* due to the ability of these policies to keep a fraction of the working set on the caches since the working set from *xalancbmk* is so small, a sizeable portion of it is able to stay on the cache and improve the final performance, mitigating the gains from HBPB.

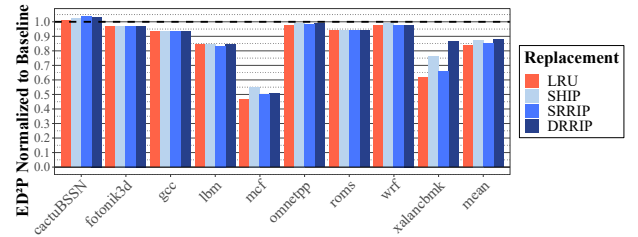


Fig. 16: Effect of HBPB on the ED²P for the SPEC CPU 2017 benchmarks with different LLC replacement policies.

LRU is the replacement policy where HBPB yielded the biggest reductions in ED²P. On average, 16.5%. With the other three policies, however, HBPB still improved ED²P, although on a smaller margin. On SHiP, the average reduction was 12.5%. On SRRIP, 15%, and on DRRIP, 12.1%. Although with some variance, HBPB performed similarly with different LLC replacement policies, still yielding considerable reductions in ED²P.

VI. CONCLUSIONS

This paper presented History-Based Preemptive Bypassing (HBPB), a novel mechanism for bypassing the cache for all memory accesses from instructions not known to be cache-friendly based on the access history. HBPB learns which instructions are cache-friendly by keeping a copy of the tag

array of the caches as if there were no bypasses and checking if the access would hit on this cache.

HBPB improves the latency of instructions that miss in the cache through the anticipation of DRAM requests and improves overall system performance by avoiding cache pollution. Avoiding unnecessary cache fills and Write Backs reduces energy waste in the caches. Improving the hit ratio reduces DRAM accesses, which further enhances energy efficiency.

Compared to a traditional cache architecture with no prefetchers, HBPB achieves reductions in execution time of up to 19.5% for a full-fledged SPEC CPU application. The mechanism also yields energy savings of up to 28.6% for a SPEC application even when accounting for its overhead. When using ED²P as a metric, HBPB improves nearly all the highly memory-intensive applications tested from SPEC 2017.

We tested HBPB when employing hardware data prefetchers, and the mechanism was proven to be either advantageous or at least not detrimental on every application when considering any metric. HBPB also still proved to be useful when considering different cache replacement policies and cache capacities.

A. Future Work

In a future work, we intend to perform an analysis of the effects of HBPB on a multicore architecture. On such a hardware, HBPB should have its capabilities magnified due to the increased pressure on the memory subsystem and competition for the caches. Eventually, the mechanism would have to be tweaked to perform better on this condition. HBPB has the potential to mitigate cache pollution from a streaming program while keeping the caches for cache-friendly applications.

REFERENCES

- [1] C.-H. Chi and H. Dietz, "Improving cache performance by selective cache bypass," in *Proceedings of the Twenty-Second Annual Hawaii Int. Conf. on System Sciences. Volume 1: Architecture Track*, vol. 1, pp. 277–278, IEEE Computer Society, 1989.
- [2] P. Guide, "Intel® 64 and ia-32 architectures software developer's manual," *Volume 2B: Instruction Set Reference*, 2020.
- [3] S. Kumar and P. Singh, "An overview of modern cache memory and performance analysis of replacement policies," in *2016 IEEE Int. Conf. on Engineering and Technology (ICETECH)*, pp. 210–214, IEEE, 2016.
- [4] A. Sandberg, D. Eklöv, and E. Hagersten, "Reducing cache pollution through detection and elimination of non-temporal memory accesses," in *SC'10: Proceedings of the 2010 ACM/IEEE Int. Conf. for High Performance Computing, Networking, Storage and Analysis*, pp. 1–11, IEEE, 2010.
- [5] J. Albericio, P. Ibáñez, V. Viñals, and J. M. Llabería, "The reuse cache: Downsizing the shared last-level cache," in *2013 46th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, pp. 310–321, IEEE, 2013.
- [6] H. J. Bae and L. Choi, "Filter cache: filtering useless cache blocks for a small but efficient shared last-level cache," *The Journal of Supercomputing*, vol. 76, no. 10, pp. 7521–7544, 2020.
- [7] J. Wang, P. Ramrakhiani, W. Elsasser, and L. K. John, "Reducing data movement and energy in multilevel cache hierarchies without losing performance: Can you have it all?," in *28th Int. Conf. on Parallel Architectures and Compilation Techniques (PACT)*, pp. 383–394, IEEE, 2019.
- [8] R. Egawa, R. Saito, M. Sato, and H. Kobayashi, "A layer-adaptable cache hierarchy by a multiple-layer bypass mechanism," in *Proceedings of the 10th Int. Symp. on Highly-Efficient Accelerators and Reconfigurable Technologies*, pp. 1–6, 2019.
- [9] M. Chaudhuri, J. Gaur, N. Bashyam, S. Subramoney, and J. Nuzman, "Introducing hierarchy-awareness in replacement and bypass algorithms for last-level caches," in *Proceedings of the 21st Int. Conf. on Parallel architectures and compilation techniques*, pp. 293–304, 2012.
- [10] W. A. Wulf and S. A. McKee, "Hitting the memory wall: implications of the obvious," *ACM SIGARCH computer architecture news*, vol. 23, no. 1, pp. 20–24, 1995.
- [11] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr, and J. Emer, "Ship: Signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM Int. Symp. on Microarchitecture*, pp. 430–441, 2011.
- [12] A. Jain and C. Lin, "Back to the future: Leveraging belady's algorithm for improved cache replacement," in *2016 ACM/IEEE 43rd Annual Int. Symp. on Computer Architecture (ISCA)*, pp. 78–89, IEEE, 2016.
- [13] N. Beckmann and D. Sanchez, "Maximizing cache performance under uncertainty," in *2017 IEEE Int. Symp. on High Performance Computer Architecture (HPCA)*, pp. 109–120, IEEE, 2017.
- [14] J. Kim, E. Teran, P. V. Gratz, D. A. Jiménez, S. H. Pugsley, and C. Wilkerson, "Kill the program counter: Reconstructing program behavior in the processor cache hierarchy," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 737–749, 2017.
- [15] A. Jain and C. Lin, "Rethinking belady's algorithm to accommodate prefetching," in *2018 ACM/IEEE 45th Annual Int. Symp. on Computer Architecture (ISCA)*, pp. 110–123, IEEE, 2018.
- [16] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *Proceedings of the 52nd Annual IEEE/ACM Int. Symp. on Microarchitecture*, pp. 413–425, 2019.
- [17] S. Gupta, H. Gao, and H. Zhou, "Adaptive cache bypassing for inclusive last level caches," in *2013 IEEE 27th Int. Symp. on Parallel and Distributed Processing*, pp. 1243–1253, IEEE, 2013.
- [18] J. Sim, J. Lee, M. K. Qureshi, and H. Kim, "Flexclusion: Balancing cache capacity and on-chip bandwidth via flexible exclusion," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 3, pp. 321–332, 2012.
- [19] J. Gaur, M. Chaudhuri, and S. Subramoney, "Bypass and insertion algorithms for exclusive last-level caches," in *Proceedings of the 38th annual Int. Symp. on Computer architecture*, pp. 81–92, 2011.
- [20] E. Teran, Z. Wang, and D. A. Jiménez, "Perceptron learning for reuse prediction," in *2016 49th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, pp. 1–12, IEEE, 2016.
- [21] D. A. Jiménez and E. Teran, "Multiperspective reuse prediction," in *2017 50th Annual IEEE/ACM Int. Symp. on Microarchitecture (MICRO)*, pp. 436–448, IEEE, 2017.
- [22] A. Sembrant, E. Hagersten, and D. Black-Schaffer, "Data placement across the cache hierarchy: Minimizing data movement with reuse-aware placement," in *2016 IEEE 34th Int. Conf. on Computer Design (ICCD)*, pp. 117–124, IEEE, 2016.
- [23] M. Kharbutli, M. Jarrah, and Y. Jararweh, "Scip: Selective cache insertion and bypassing to improve the performance of last-level caches," in *2013 IEEE Jordan Conf. on Applied Electrical Engineering and Computing Technologies (AEECT)*, pp. 1–6, IEEE, 2013.
- [24] M. Jalili and M. Erez, "Reducing load latency with cache level prediction," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 648–661, IEEE, 2022.
- [25] R. Bera, K. Kanellopoulos, S. Balachandran, D. Novo, A. Olgun, M. Sadrosadati, and O. Mutlu, "Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction," *arXiv preprint arXiv:2209.00188*, 2022.
- [26] ChampSim, "Champsim." Retrieved November 18, from <https://github.com/ChampSim/ChampSim>, 2021.
- [27] J. Henning, "Spec cpu 2017@." Retrieved February 12, 2021 from <https://www.spec.org/cpu2017/Docs/overview.html#benchmarks>, 2021.
- [28] Q. Duon, "ML-based data prefetching competition." Retrieved September 12, 2021 from <https://sites.google.com/view/mlarchsys/isca-2021/ml-prefetching-competition>, 2021.
- [29] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [30] A. Jaleel, K. B. Theobald, S. C. Steely Jr, and J. Emer, "High performance cache replacement using re-reference interval prediction (rip)," *ACM SIGARCH Computer Architecture News*, vol. 38, no. 3, pp. 60–71, 2010.